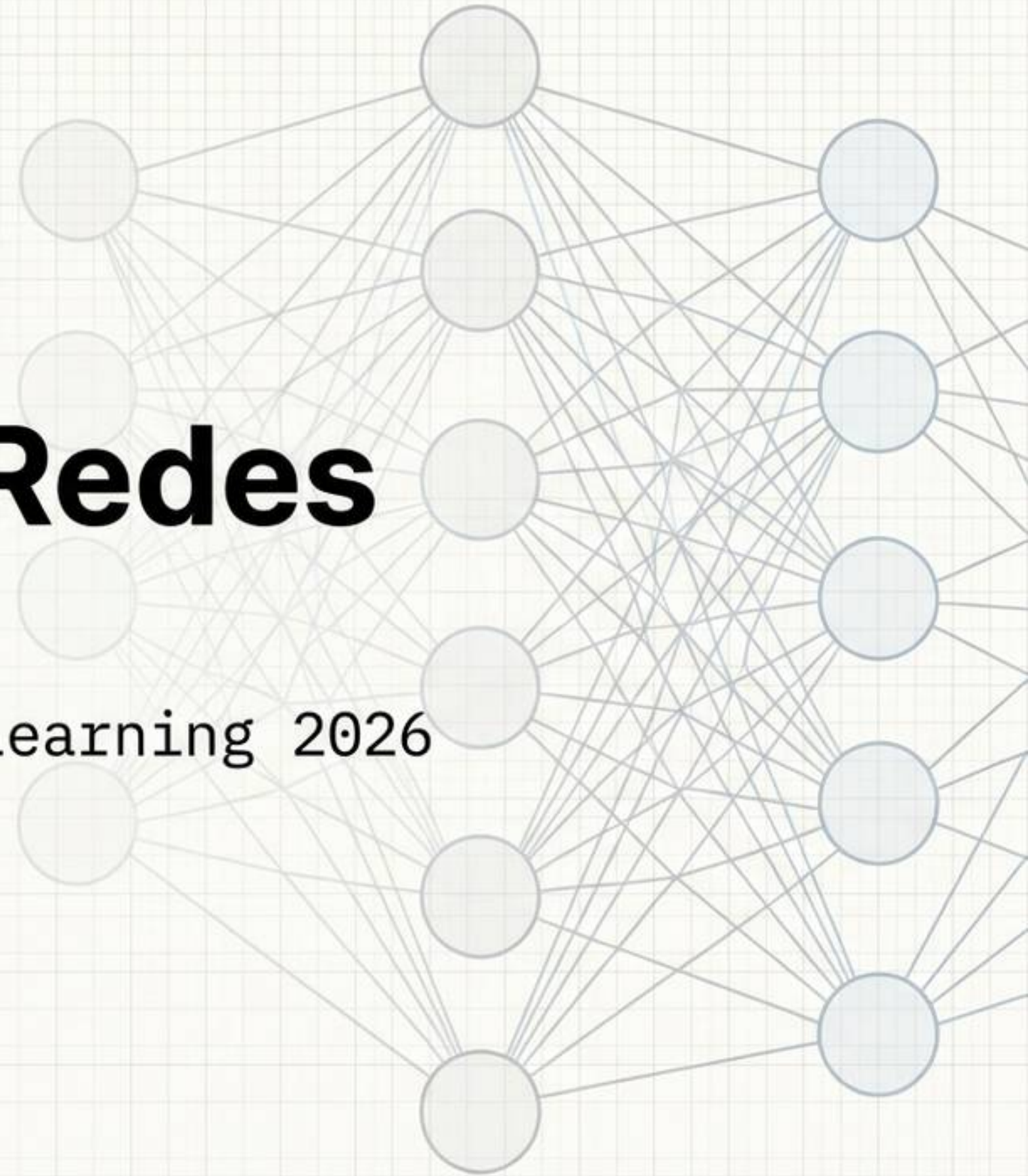


# Introducción a Redes Neuronales

Semana 4 | Curso de Machine Learning 2026

Dr. Julio Lopez-Nunez



# Cognitive Blueprint: The Neural Network Model

HIGH-LEVEL FLOW



DETAILED ZOOM: RED NEURONAL ARTIFICIAL

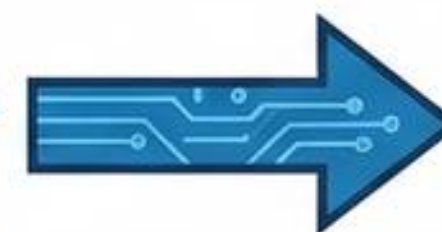
Una **red neuronal artificial** es una implementación concreta del componente modelo. Aprende relaciones complejas entre datos de **entrada** y **resultados esperados** ajustando parámetros progresivamente.



[Imagen]



[Red Neuronal]



[Predicción: Perro]



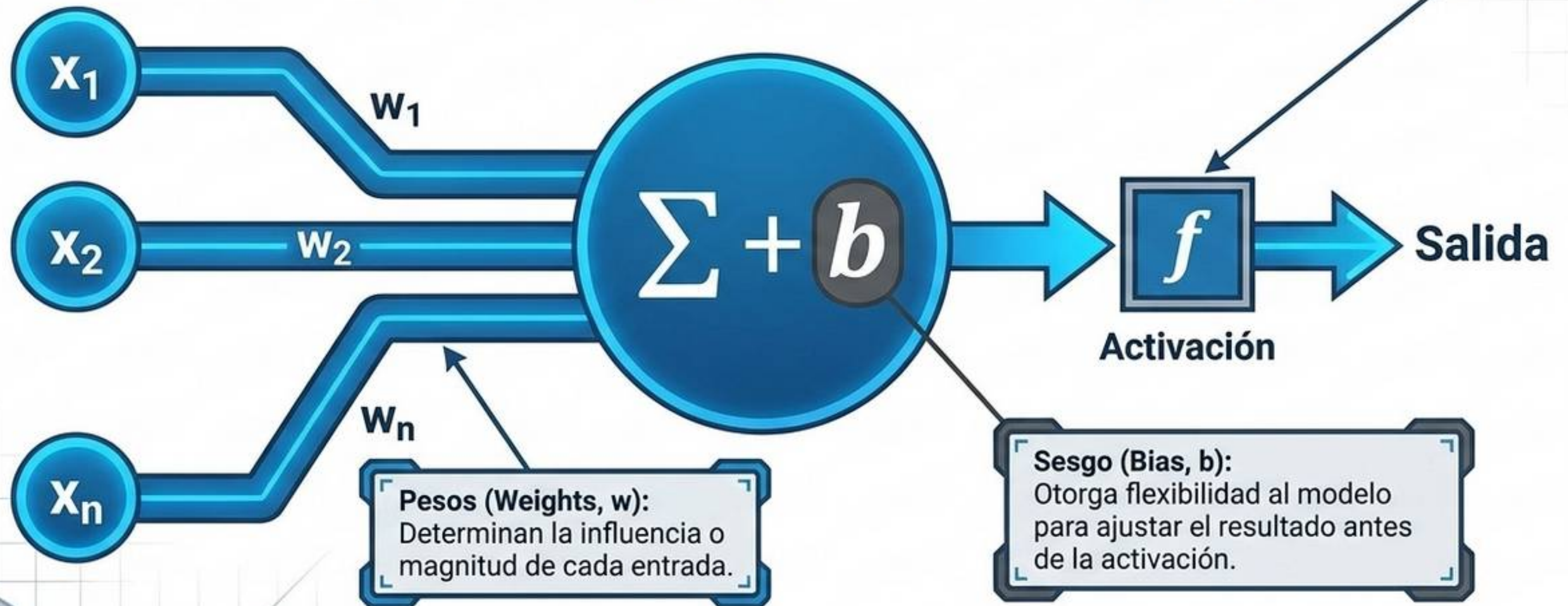
( $\leftrightarrow$ ) se compara con

[Etiqueta Real]



# La Neurona Artificial: Unidad Fundamental

$$Z = (w_1x_1 + w_2x_2 + \dots + w_nx_n) + b$$

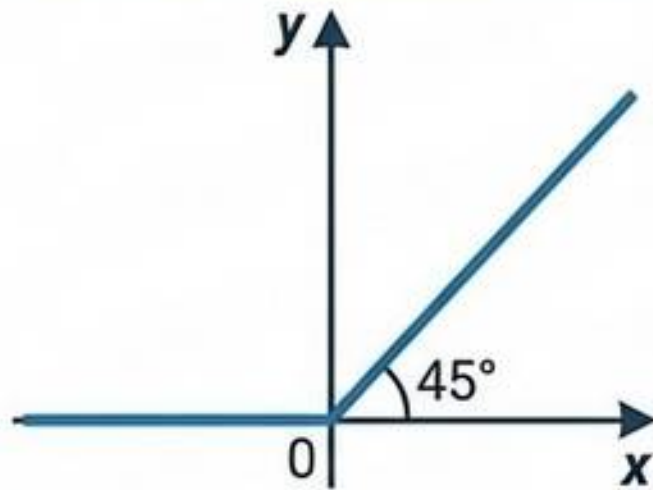


**Nota:** Ambos ( $w$  y  $b$ ) son los parámetros aprendidos por la red.

# Funciones de Activación: Transformación No Lineal

Introducen **no linealidad**, permitiendo a la red aprender relaciones complejas imposibles de modelar con matemáticas puramente lineales.

## ReLU (Rectified Linear Unit)

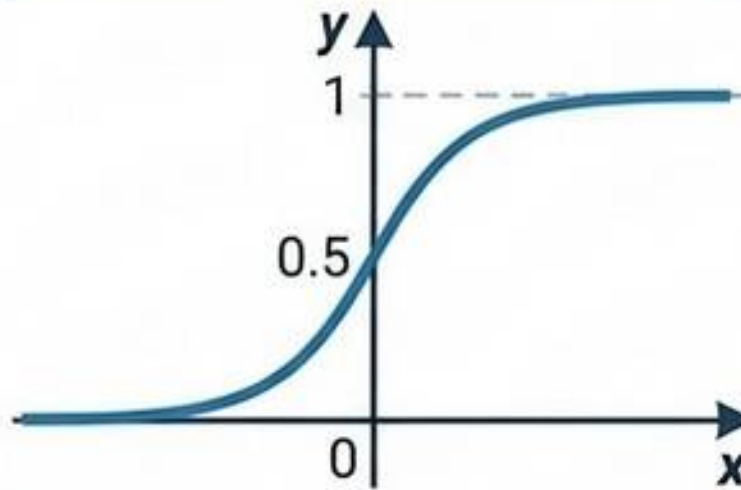


**Fórmula:**  $\max(0, x)$

**Lógica:** Negativo  $\rightarrow$  0;  
Positivo  $\rightarrow$  Mantiene valor.

**Uso:** Ampliamente usada en capas internas/ocultas.

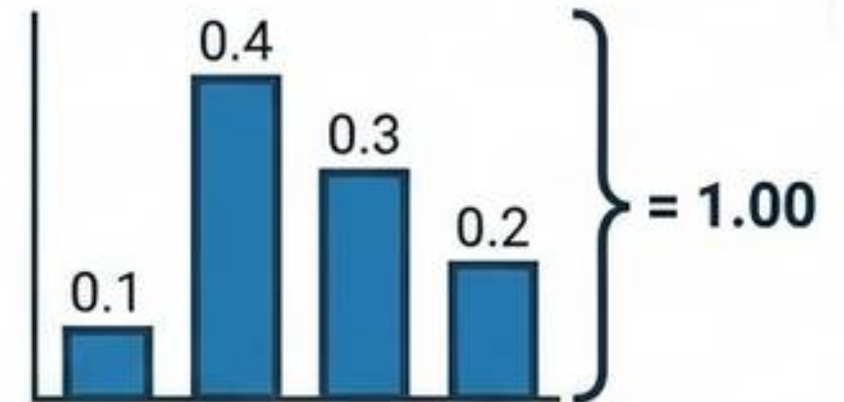
## Sigmoid



**Lógica:** Comprime cualquier valor entre 0 y 1.

**Uso:** Ideal para clasificación binaria (ej. Perro vs. Gato).

## Softmax



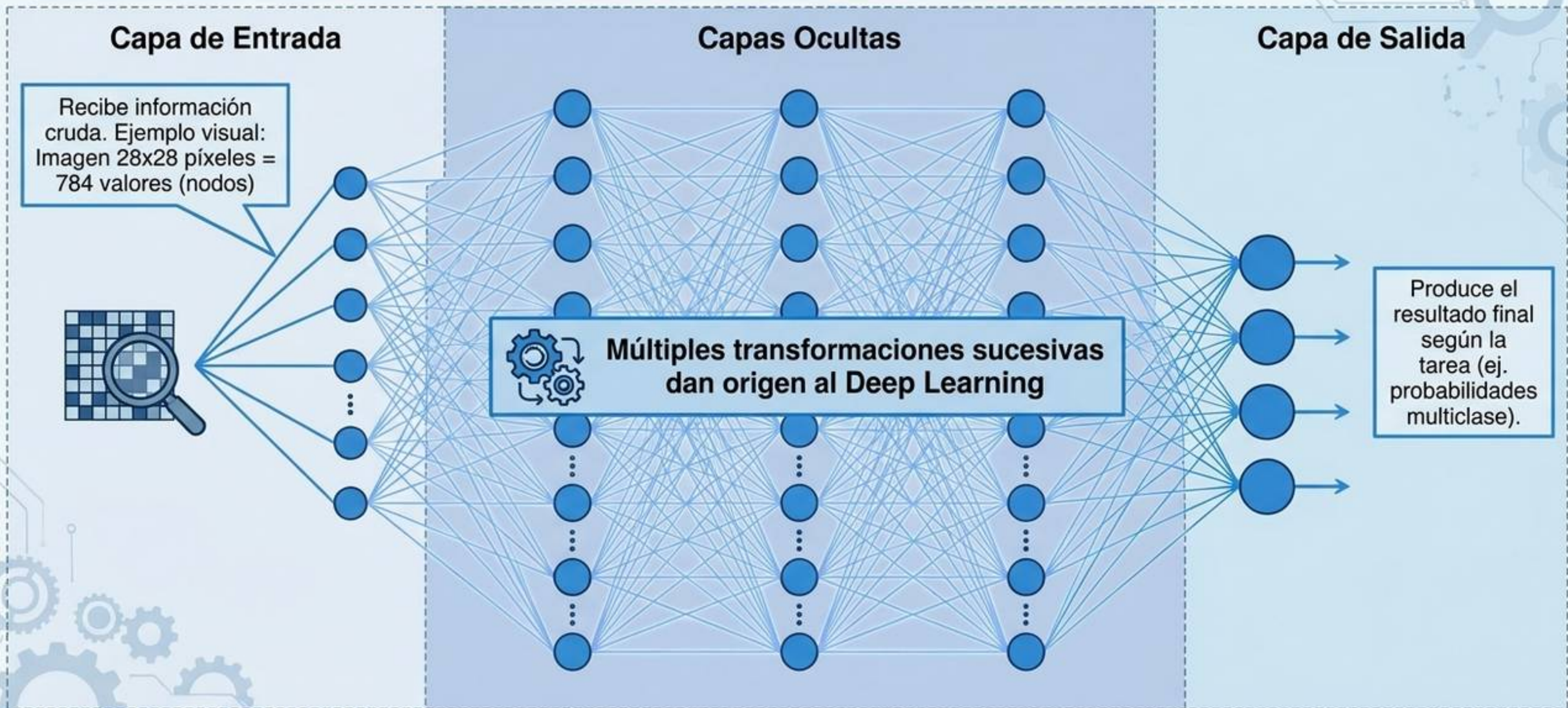
**Lógica:** Transforma salidas en probabilidades proporcionales (suma total = 1.00).

**Uso:** Capa de salida para múltiples clases mutuamente excluyentes (multiclase).

**Nota:** La elección de la función de activación depende del tipo de problema y la arquitectura de la red.



# De la Neurona a la Red: Capas y Deep Learning



**Nota:** Una sola neurona tiene capacidades limitadas; agruparlas permite modelar representaciones progresivamente complejas.

# Forward Propagation & Función de Pérdida



**Forward Propagation:** La información viaja hacia adelante hasta producir una salida usando pesos, sesgos y activaciones.

## DIAGNOSTIC



$$[\text{Predicción}] \leftrightarrow [\text{Etiqueta Real}] = [\text{Función de Pérdida (Loss)}]$$

Cuantifica la diferencia entre la predicción generada y el resultado esperado correcto. El objetivo del entrenamiento es minimizar esta función.

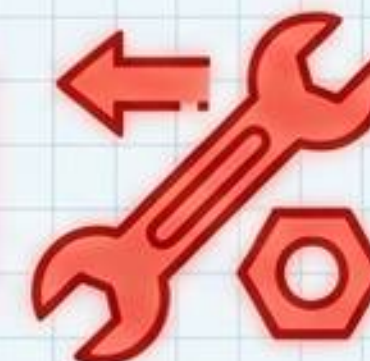


# Backpropagation: Retropropagación del Error

**Hacia adelante → Producir predicción**



**Hacia atrás ← Aprender del error**



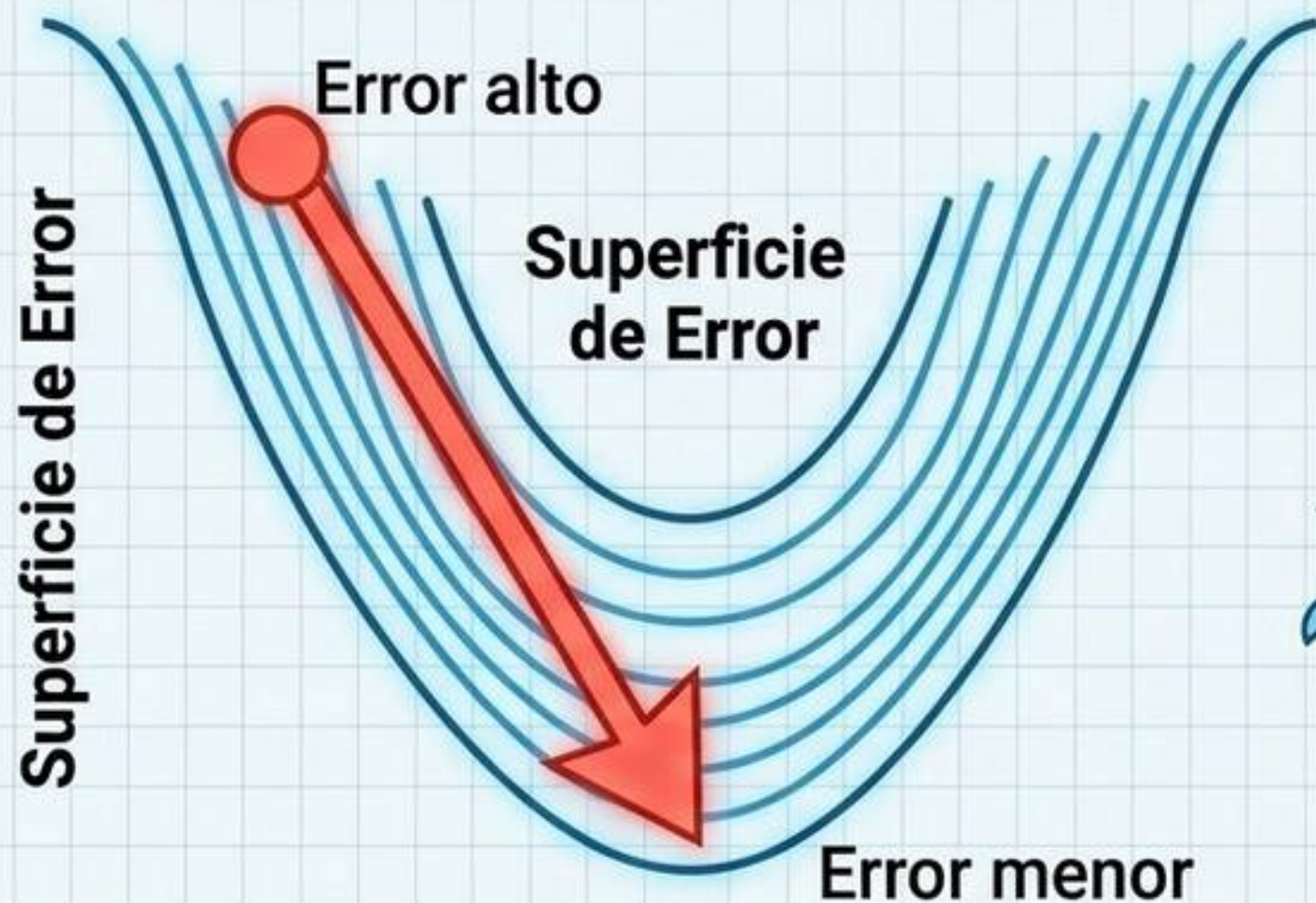
Procedimiento mediante el cual la información del error (pérdida) viaja hacia las capas anteriores.

Determina cómo contribuyó cada parámetro al valor de la función de pérdida (mediante derivadas / regla de la cadena).

Indica la dirección y magnitud en que deben modificarse los pesos y sesgos.



# Optimización y Descenso de Gradiente



**Descenso de Gradiente:** Búsqueda direccional para encontrar la zona de menor error.

## Optimizadores

Algoritmos (SGD, Adam, RMSprop) que ejecutan la actualización de parámetros basándose en Backpropagation.

## Tasa de Aprendizaje (Learning Rate)

Define el tamaño de los pasos o ajustes:

- **Muy grande:** Salta el valle; dificulta convergencia.
- **Muy pequeño:** Descenso excesivamente lento.



# Configuración del Modelo: Parámetros vs. Hiperparámetros

## Parámetros



**Origen:** Aprendidos automáticamente por el modelo.



**Qué hacen:** Transforman los datos internamente.



**Ejemplos:** Pesos ( $w$ ), Sesgos ( $b$ ).

## Hiperparámetros



**Origen:** Configuraciones definidas por el humano antes del entrenamiento.



**Qué hacen:** Controlan el proceso de aprendizaje y la arquitectura.



**Ejemplos:** Tasa de aprendizaje (Learning rate), Cantidad de neuronas/capas, Optimizador.

## Conceptos de Iteración (Hiperparámetros Clave)



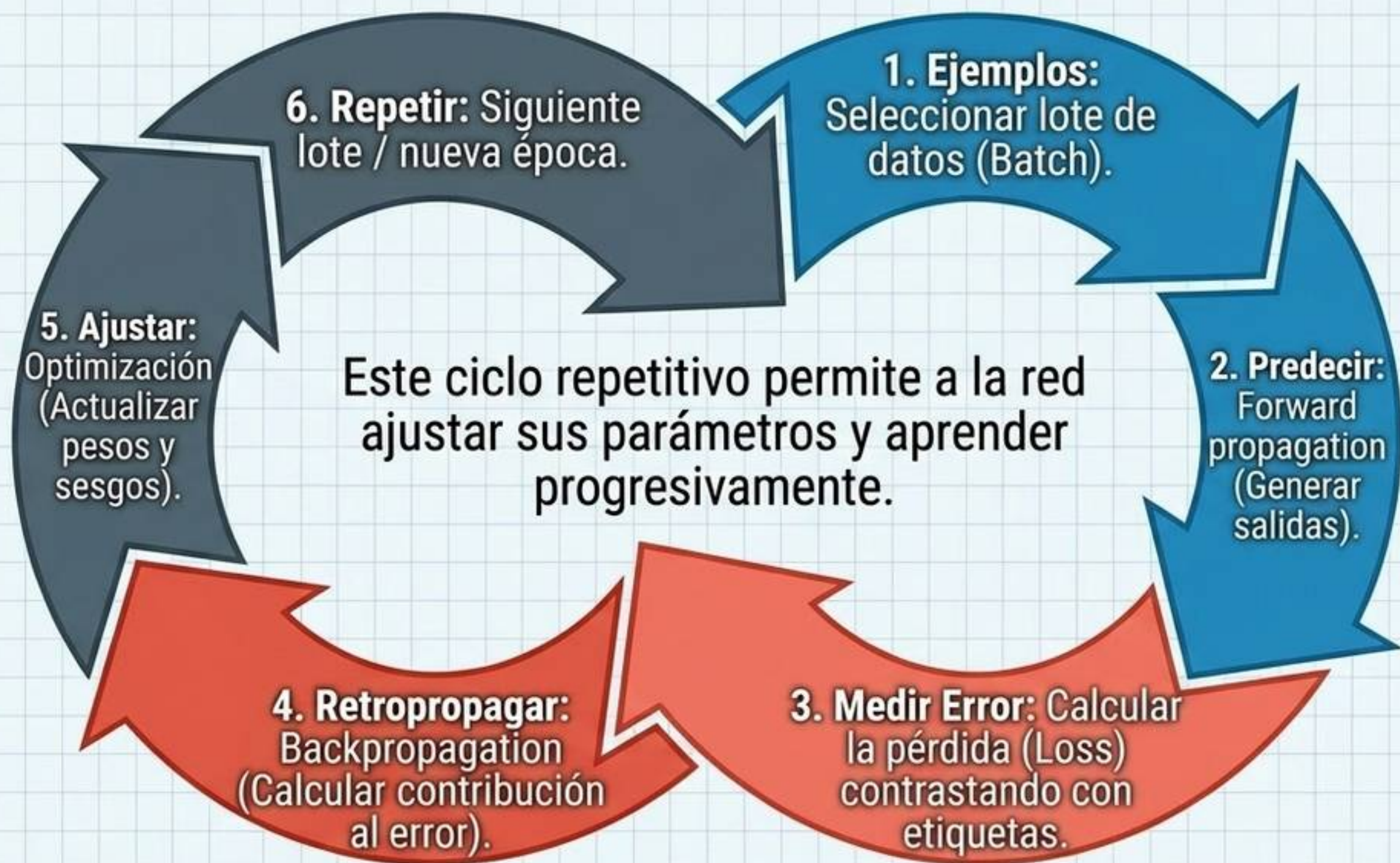
**Época (Epoch):** Una pasada completa de todo el Dataset por la red.



**Lote (Batch):** Subgrupos más pequeños de datos procesados en cada iteración dentro de una época.

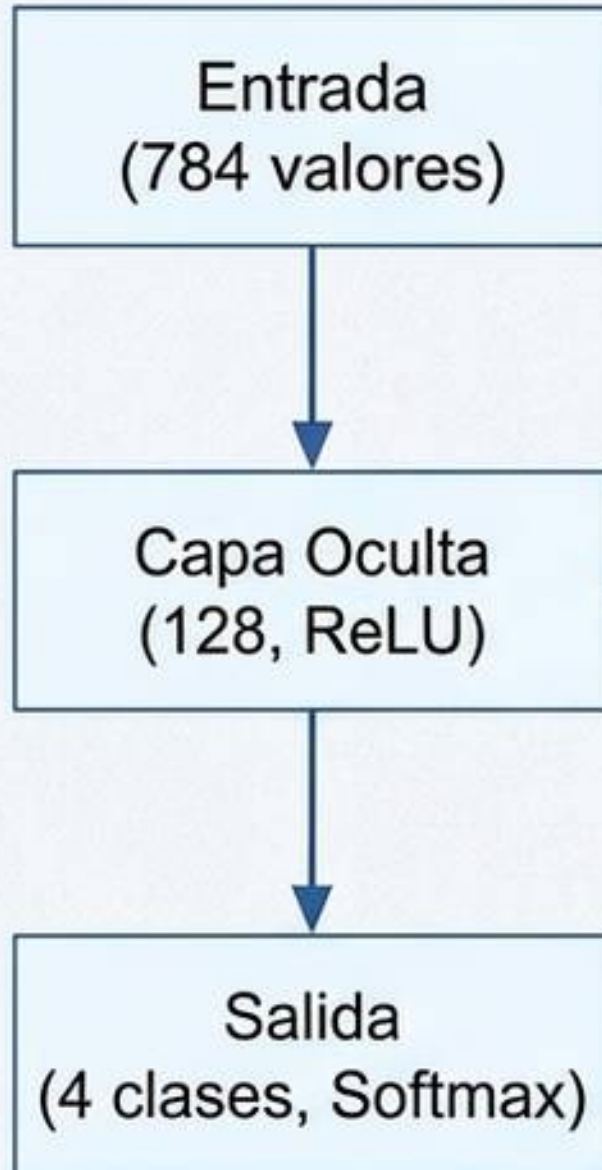


# El Ciclo Completo de Entrenamiento



# Cognitive Blueprint

## Del Concepto al Código (Python)



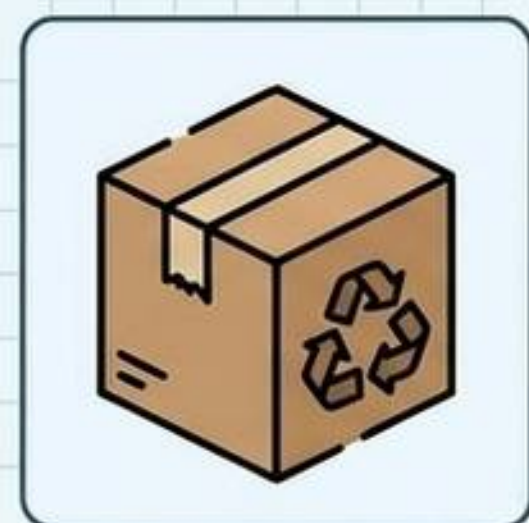
```
# Arquitectura
modelo = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(4, activation='softmax')
])

# Configuración del entrenamiento
modelo.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

(Optimizador)



# Aplicación al Proyecto Integrador: Clasificación de Residuos



[Miniatura de  
Cartón]

[Red Neuronal  
Básica  
(Prototipo)]

Probabilidades Softmax (Suma = 1.00)

Plástico: 0.06 (6%)

Vidrio: 0.09 (9%)

**Cartón: 0.81 (81%) → Predicción Ganadora**

Metal: 0.04 (4%)

Nota: Esta red básica es nuestro primer prototipo para comprender el entrenamiento. Posteriormente evolucionará a una Red Neuronal Convolutiva (CNN).



# Repaso Clínico: Preguntas para Discusión

1 ¿Qué función cumplen los pesos en una neurona con cinco entradas?

2 ¿Qué activación usamos para clasificar 4 categorías en la salida: ReLU o Softmax? ¿Por qué?

3 ¿Qué elementos traducen el error de predicción en ajustes matemáticos?

4 Un modelo usa epochs=20. ¿Qué significa conceptualmente?

5 ¿Adam es un peso, una activación o un optimizador?

6 Al usar un modelo ya entrenado en una app, ¿se ejecuta backpropagation o inferencia?



# Síntesis de la Semana 4

- ✓ Una red neuronal agrupa neuronas en capas (entrada, ocultas, salida).
- ✓ Cada neurona combina entradas con pesos y sesgos (parámetros), y aplica una función de activación (no linealidad).
- ✓ Forward propagation genera predicciones; la función de pérdida cuantifica el error respecto a la etiqueta.
- ✓ Backpropagation determina cómo contribuye cada parámetro al error.
- ✓ El optimizador (ej. Adam) ajusta los parámetros en iteraciones (lotes y épocas).
- ✓ Los hiperparámetros (tasa de aprendizaje, épocas) son configurados, no aprendidos.
- ✓ Este prototipo prepara el terreno para las Redes Convolucionales (CNN) del proyecto final.

# Próximos Pasos: Hacia la Semana 5



**Cierre de Unidad 1 (El Modelo). Iniciamos Unidad 2 (La Materia Prima).**

**¿Qué datos necesitamos y cómo debemos prepararlos para que el modelo aprenda correctamente?**